

Midway Check-in

Zilin Dai

CS265, Spring 2025

1 Introduction

In this milestone I built a computational graph profiler for PyTorch that instruments a full training iteration (forward pass, backward pass, and optimizer step). The profiler runs the traced graph node by node, measures each operator's execution time and memory footprint, classifies every tensor as a parameter, gradient, activation, optimizer state, or other, and performs static analysis on activations to determine when they are created and when they are actually needed. I ran experiments on ResNet-152 (a vision model) and BERT-base (a language model) at batch sizes 2, 4, 8, and 16. The goal is to collect the data needed for the activation checkpointing algorithm in the next phase: specifically, which activations to keep and which to recompute.

2 Problems Tackled

- **[Region Classification]** The traced graph is one flat sequence of nodes covering forward, loss, backward, and optimizer. We need to figure out which region each node belongs to so we can identify activations (forward nodes used in backward).
- **[Tensor Type Categorization]** Each tensor flowing through the graph needs to be labeled as one of: parameter, gradient, activation, optimizer state, or other. Without this, we cannot tell what is taking up memory or what is safe to discard.
- **[Activation Lifetime Analysis]** For each activation, we need to know when it was created (forward pass), when it was last used in the forward pass, and when it is first needed in the backward pass.
- **[Per-Node Timing and Memory Measurement]** We need actual runtime and memory numbers for each operator. The checkpointing algorithm trades compute for memory, so it needs to know how expensive it is to recompute an activation versus how much memory it saves.
- **[Peak Memory Simulation]** We need to track how much memory is alive at each point during execution and find where the peak occurs, broken down by tensor type. This tells us how much memory activation checkpointing could save.

3 Technical Description

3.1 Region Classification

a) Problem framing: The compiler in `graph_tracer.py` traces the entire training step (forward, loss, backward, optimizer) into a single flat `fx.Graph`. There are no explicit boundaries. But to identify activations we need to know which nodes are in the forward pass and which are in the backward pass, because an activation is defined as a tensor produced in the forward pass that is consumed in the backward pass.

b) High-level solution: The starter code inserts sentinel nodes via `SEPFunction`. `torch.ops.separator.sep` marks the end of the forward pass, and `torch.ops.separator.sep_backward` marks the start of the backward pass. The optimizer step starts at the `torch.ops.aten._fused_adam` node (when using fused Adam). I scan the node list once, find these three indices, and assign every node to one of four regions: forward (up to `sep`), loss (between `sep` and `sep_backward`), backward (between `sep_backward` and `fused_adam`), or optimizer (`fused_adam` onward). When `_fused_adam` is not present (e.g., the starter code uses `foreach=True`), I fall back to detecting the first `_foreach_*` operation after `sep_backward` as the optimizer boundary.

c) Deeper details: The region assignment is a single linear pass over the node list in `__init__`. Each node gets stored in a `node_region` dict. This runs once at profiler construction time, not during execution, so it adds no overhead to the profiling loop. The regions are used downstream by the activation identification logic and the memory breakdown.

3.2 Tensor Type Categorization

a) Problem framing: The project spec requires classifying every tensor as parameter, gradient, activation, optimizer state, or other. This matters because each type has different lifetime behavior—parameters live for the whole iteration, activations are created in forward and freed in backward, gradients are created in backward and consumed by the optimizer. Without this classification we cannot produce the required memory breakdown.

b) High-level solution: The `_fused_adam` node gives us everything directly. Its arguments are:

- `args[0]`: list of parameter nodes
- `args[1]`: list of gradient nodes
- `args[2]` through `args[5]`: optimizer state nodes (`exp_avg`, `exp_avg_sq`, `max_exp_avg_sq`, step counters)

For activations, I look for nodes that are (1) in the forward region, (2) are `call_function` or `call_method` ops (not placeholders), and (3) have at least one user node in the backward region. The `_fused_adam` *outputs* are also classified: it returns (`updated_params`, `updated_exp_avgs`, `updated_exp_avg_sqs`), so the `getitem` nodes extracting from its output are labeled `PARAM` (index 0) or `OPT_STATE` (indices 1–2). Everything else is labeled `OTHER`.

c) Deeper details: I store the classification in a `node_type_map` dict mapping each node to a `NodeType` enum (PARAM, GRAD, ACT, OPT_STATE, OTHER). The activation identification iterates over forward-region nodes and checks their users’ regions. This is $O(nodes \times avg_users)$, which in practice is fast since most nodes have 1–3 users. The classification is done once at construction and reused during every profiling run.

3.3 Activation Lifetime Analysis

a) Problem framing: The checkpointing algorithm needs to decide which activations to keep and which to recompute. To make this decision, it needs to know: when each activation is produced, how long it sits idle in memory, and when it is actually consumed. An activation with a long idle span and small recomputation cost is a good candidate for checkpointing.

b) High-level solution: For each activation node, I record four indices: `created_at` (where it is produced in forward), `last_fwd_use` (last node in forward that reads it), `first_bwd_use` (first node in backward that reads it), and `last_bwd_use` (last backward consumer). The idle span is `first_bwd_use - last_fwd_use`. I iterate over each activation’s user set and check the region of each user.

c) Deeper details: The output is a table with one row per activation showing all four indices, the idle span, and the tensor size in MB. For ResNet-152 at `bs=4`, there are 777 activations. The first convolution output has an idle span of 2592 nodes—it is created at node 2337 and not needed until node 4931. For BERT at `bs=4`, there are 355 activations; the first residual connection (`add_1`, 1.5 MB) has an idle span of 2040 nodes. In both models, earlier-layer activations have the longest idle spans because backward processes layers in reverse order.

Example from ResNet-152 (`bs=4`):

Name	Created	LastFwd	1stBwd	LastBwd	Idle	MB
convolution	2337	2339	4931	4931	2592	12.250
relu_	2344	2345	4929	4930	2584	12.250
getitem_5	2347	2347	4929	4929	2582	6.125
convolution_1	2348	2350	4917	4917	2567	6.125

Example from BERT (`bs=4`):

Name	Created	LastFwd	1stBwd	LastBwd	Idle	MB
expand_1	806	808	2869	2869	2061	0.004
add_1	811	812	2852	2852	2040	1.500
getitem_4	818	818	2851	2851	2033	0.375
view	839	841	2844	2844	2003	1.500

3.4 Per-Node Timing and Memory Measurement

a) Problem framing: We need actual numbers for how long each operator takes and how much memory its output consumes. The profiler spec requires this data for every operator in the graph.

b) High-level solution: I wrap each node’s execution with `torch.cuda.Event` pairs for timing. After execution, I compute the output tensor’s memory as `element_size × nelement`. Both are collected over 3 profiling iterations (after 2 warmup iterations to avoid cold-start effects) and averaged.

c) Deeper details: The timing uses `start.record()` before `super().run_node(n)` and `end.record()` after, followed by `torch.cuda.synchronize()` and `start.elapsed_time(end)`. The synchronize after each node serializes execution, which is necessary for accurate per-node timing but does slow down the profiling run compared to normal execution. For memory, `_tensor_mem` handles both single tensors and nested tuples/lists (since some ops like `_fused_adam` return tuples of tensors).

Results for ResNet-152 (bs=4):

Region	Nodes	Time (ms)	%
Forward	3635	78.8	26.6
Backward	2235	98.2	33.1
Optimizer	2807	119.4	40.3
Total	8679	296.5	

Results for BERT (bs=4):

Region	Nodes	Time (ms)	%
Forward	1538	77.7	32.5
Backward	1726	82.6	34.5
Optimizer	1187	78.8	32.9
Total	4453	239.1	

Tensor memory by type:

Type	ResNet-152 (MB)	BERT (MB)
Parameters	229.6	414.3
Gradients	229.6	414.3
Opt States	459.2	828.5
Activations	683.0	713.3

3.5 Peak Memory Simulation

a) Problem framing: We need to know the peak memory during a training iteration and what types of tensors contribute to it. This is the baseline that activation checkpointing aims to reduce.

b) High-level solution: I maintain an “alive” set of tensors during graph execution. When a node produces a tensor, I add its size to the set. When all consumers of a tensor have executed, I remove it. At each step I sum the alive set and record the total, broken down by tensor type. The peak is the max across all steps. I track two peaks: the overall peak (across all regions) and the forward+backward peak (excluding the optimizer region), since activation checkpointing targets the forward/backward memory, not the optimizer.

c) Deeper details: I precompute `last_user[n]` for every node—the last node in topological order that reads `n`. In `run_node`, after executing node `n`, I check each of `n`’s inputs: if `n` is that

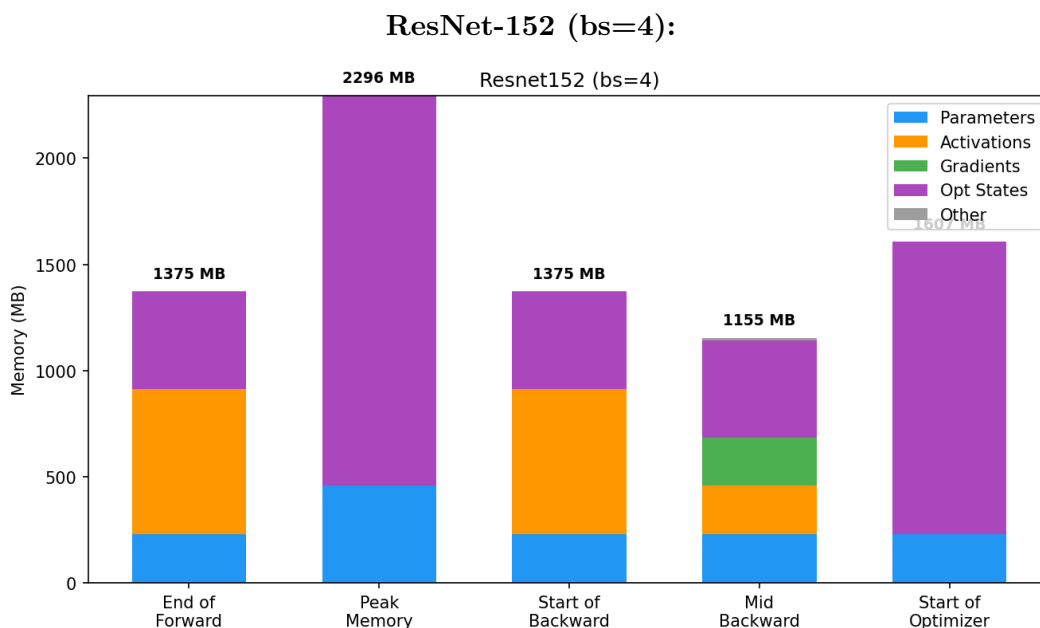
input’s last user, I remove it from the alive set. This mirrors how `fx.Interpreter` garbage-collects values, so the simulation is consistent with actual execution.

Forward+backward peak memory and breakdown (bs=4):

	ResNet-152	BERT
Fwd+Bwd Peak	1423 MB	1967 MB
Parameters	230 MB (16.1%)	414 MB (21.1%)
Activations	657 MB (46.2%)	712 MB (36.2%)
Gradients	63 MB (4.4%)	0 MB (0.0%)
Opt States	459 MB (32.3%)	829 MB (42.1%)
Other	14 MB (1.0%)	13 MB (0.6%)

Activations make up 46% of the forward+backward peak for ResNet-152 and 36% for BERT. This is the memory that activation checkpointing can reduce by recomputing instead of storing.

Memory breakdown at key execution points:



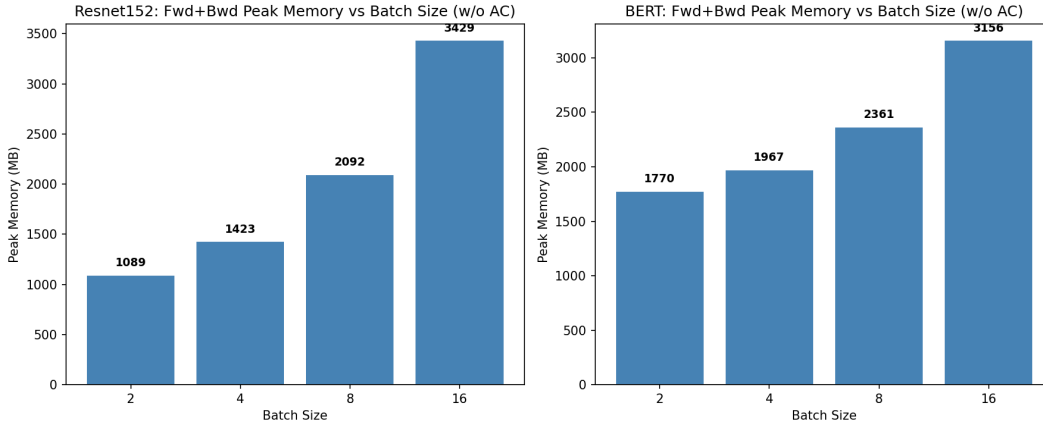
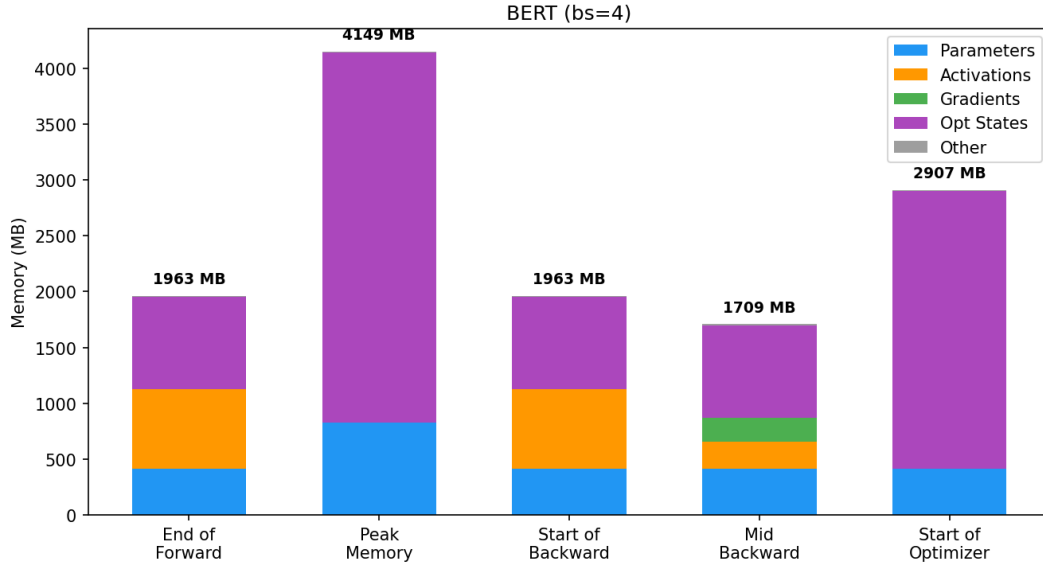
These charts show how memory composition changes across the training iteration. At end-of-forward, activations (orange) are the largest variable-cost component. By mid-backward they are being freed as gradients (green) are computed. At the optimizer step, activations are gone entirely. The “Peak Memory” bar shows the overall peak at the `_fused_adam` node, where the optimizer outputs all updated parameters and states simultaneously alongside the still-alive inputs—effectively doubling the parameter and optimizer state footprint at that instant.

Peak memory vs batch size (forward+backward only):

Model	bs=2	bs=4	bs=8	bs=16
ResNet-152	1,089	1,423	2,092	3,429
BERT	1,770	1,967	2,361	3,156

Both models show roughly linear growth with batch size. ResNet-152 roughly triples from bs=2

BERT (bs=4):



to bs=16 because its activations (feature maps at each conv layer) scale directly with batch size. BERT’s growth is less steep because a larger fraction of its memory is fixed costs (parameters + optimizer states), but the trend is clear. This is the memory that activation checkpointing will reduce in Phases 2 and 3.

4 Challenges

- **Overall peak landing on the optimizer.** The overall peak in the simulation happens at the `_fused_adam` node, not during forward/backward. This is because `_fused_adam` outputs all updated parameters and optimizer states as one large tuple—at that instant, both the old inputs and new outputs are alive, roughly doubling the parameter + optimizer state footprint. Initially, these output tensors were all classified as “other” since the type categorization only looked at `_fused_adam`’s *inputs* (`args[0]–[5]`). I fixed this by also classifying the *output* `getitem` nodes: `_fused_adam` returns (`updated_params`, `updated_exp_avgs`, `updated_exp_avg_sqs`), so index 0 maps to PARAM and indices 1–2 map to OPT_STATE. To get meaningful numbers for the “peak memory vs batch size” graph, I report the for-

ward+backward peak separately, since that is what activation checkpointing actually targets.

- **Memory simulation vs actual CUDA memory.** The simulation tracks tensor-level memory (`element_size` $\times$ `num_elements`) but does not account for CUDA allocator overhead, memory fragmentation, or temporary buffers inside operators (e.g., cuDNN workspace for convolutions). The real GPU memory usage is higher. For this project the simulation is sufficient because we need the type-level breakdown, which only the simulation can provide.
- **Connecting profiler output to the checkpointing algorithm.** The profiler now produces activation sizes, idle spans, and recomputation costs (operator runtimes), which Phase 2 needs as input to the μ -TWO algorithm. The main challenge will be mapping from the profiler’s node-level data (flat graph) to the subgraph extraction needed for Phase 3—identifying which nodes to recompute and inserting them into the backward pass.